

Reinforcement Learning Integration for ISAC Power-Waveform Allocation

Executive Summary

The current ISAC power-waveform allocation project formulates a weighted sensing/communication objective (detection probability vs. data rate) optimized by metaheuristics (GWO, PSO, etc.) and analyzed via XGBoost surrogate models. Expanding this with *Reinforcement Learning* (RL) would recast allocation as a Markov decision process, enabling an agent to learn sequential allocation policies under time-varying channels [36†L66-L74] [9†L37-L44]. In this approach, at each time step the agent observes the current channel state (subcarrier gains for communication and sensing), selects a continuous action (power and waveform profile across subcarriers), and receives a reward based on the weighted objective. We propose designing a Gym-like environment for simulation, selecting candidate RL algorithms (e.g. SAC, PPO, TD3) suited to continuous actions, and integrating with existing simulation code. A detailed training pipeline with replay buffers, evaluation metrics, and baselines (current optimizers) will be established. We also outline rigorous debugging/monitoring practices (TensorBoard, seeding) and risk mitigations (stable algorithms, reward shaping). A multi-phase roadmap breaks the work into well-defined milestones, each with estimated duration, culminating in a minimally viable RL prototype. We provide example pseudocode, a recommended code structure, and reference key libraries and primary sources.

Required information from the second paper: To implement RL, we must extract from the current project (second paper) the exact problem details: the **state space** (e.g. how channel conditions are represented per subcarrier), the **action space** (dimensions and constraints on power and waveform profiles), the **reward/objective formula** (the weighted sum $J(p, w)$ including scaling factors and weight α), episode length (e.g. 20 time steps in the dynamic study), and any dynamics (how channels update over time). We also need model parameters (number of subcarriers, power budget, per-step channel distribution models) and evaluation metrics used (e.g. average detection probability, rate, weighted score). This information will guide the RL environment design (state/action dimensions, reward function, terminal conditions).

1. RL Problem Formulation

We formulate the ISAC allocation as a Markov Decision Process (MDP) [36†L66-L74] [16†L84-L93]. Each *time step* corresponds to a channel coherence interval (or slot) with updated channel conditions. The **state** includes all observable information the agent needs: at minimum, the per-subcarrier channel gains for communication and sensing. For example, the state vector s could concatenate the comms channel gains and sensing gains (dimensionality $2K$ for K subcarriers), possibly augmented with scenario metadata or previous action if needed. The **action** is the allocation decision: typically a continuous vector combining (1) per-subcarrier power levels p_k (with $\sum p_k = P_{\text{tot}}$) and (2) subcarrier waveform weights w_k (with $\sum w_k = K$). This yields a $2K$ -dimensional continuous action, subject to linear constraints. (If implementing RL, one may normalize actions via e.g. a Dirichlet distribution [36†L66-L74] or represent relative allocations to enforce constraints.)

The **reward** r_t at each step is designed to reflect the project's objective function. The second paper defines a weighted sum: $J(p,w) = \alpha R(p,w) + (1-\alpha)\gamma P_d(p,w)$, where R is communication rate, P_d is detection probability, α is a trade-off weight, and γ scales the sensing term [?] (we must extract these constants from the paper). We propose using the instantaneous $J(p_t, w_t)$ as the reward at each step. (Alternatively, one could give separate rewards or shape them, but the simplest is $r_t = J$.) The agent's goal is to maximize cumulative (discounted) reward $\sum_t \gamma^t r_t$ over the episode [16:L84-L93] .

An **episode** ends after a fixed horizon T (e.g. 20 slots as in the dynamic study of the paper). The terminal condition is simply $t=T$; there may be no other absorbing state (no special fail state unless one models target loss). At episode end the environment resets (new random channel realization). The discount factor γ (in RL) should be set to 1.0 if focusing on finite-horizon sum, or slightly below 1 if long-term discounting is desired.

Missing details from the paper: We must confirm from the second paper: (a) the exact state features (e.g. statistical channel parameters, K , etc.) and whether the channel evolution has memory; (b) constraints on p, w and how to encode them; (c) numeric values of α, γ used; (d) episode length and whether dynamic channel updates use a specific model. Without inventing, we note these and would extract them to precisely define the MDP.

2. Environment Design / Simulator

We will implement a custom RL environment (e.g. following the Gymnasium API [32:L149-L157]). The environment encapsulates the ISAC system: it simulates channel conditions and computes rewards. At reset, it initializes a scenario (e.g. choosing a propagation model like UMi_NLOS or others, and generating initial channel gains). Each step(*action*) call takes the agent's (p, w) allocation, computes the resulting communication rate and detection probability (using the project's formulas for SNR and detection mapping [?]), then returns `(next_state, reward, done, info)`. The *next_state* includes the new channel gains (for dynamic channels, we update them according to a specified model – e.g. independent draw per slot or a correlated fading model). If the second paper uses a finite set of 20 fixed snapshots (Table 3 mentions 4 scenarios $\times$ 20 snapshots = 80), we may randomize similarly (random draw from those scenarios).

Specifically, the environment must incorporate:

- **Channel model:** Use the same OFDM link and sensing models as the paper. For each subcarrier, generate complex channel gains for the user and the target return. If code or parameters exist (e.g. ITU channel parameters for UMi, PedB, etc.), reuse them; otherwise assume Rayleigh fading with given path loss.
- **Sensing model:** Compute the effective sensing SNR and then $P_d = Q(\Phi^{-1}(1-P_{fa})) - \sqrt{2 G \cdot \text{SNR}}$ as given in the paper's Equations (4–5). (P_{fa} and G should be taken from the paper's Table 1.)
- **Reward calculation:** Apply the weighted objective formula. Ensure to include any scaling factors (e.g. $\gamma=10$ used in experiments [?]).

- **Action enforcement:** Ensure actions respect constraints. For example, after the agent outputs a raw vector, project it onto $\{p_k \geq 0, \sum p_k = P_{\text{tot}}\}$ and similarly for w . A Dirichlet output layer (as in [36]) can naturally enforce normalization.
- **Observations:** Return the current channel gains; one might also include remaining power budget or time-step index if needed.

This simulator can be built on CPU (Python) since the computations are relatively cheap (simple formulas per step). We should design it modularly (a class `ISACEnv` with methods `reset()` and `step(action)`). Using Gymnasium's interface [32†L149-L157] ensures compatibility with RL libraries.

```

flowchart LR
    Agent[RL Agent] -->|action: power & waveform allocations| Env[ISAC Environment]
    Env -->|next state: updated channel gains| Agent
    Env -->|reward: J(p,w) = αR + (1-α)P_d| Agent

```

Figure: *System architecture*. The RL **Agent** selects an action (power vector p and waveform profile w), the **Environment** (wireless channel simulator) returns the next state (channel gains) and reward (weighted detection/communication metric) [16†L84-L93] [36†L66-L74]. The agent then updates its policy based on the reward and new state.

3. Algorithm Selection and Hyperparameters

We compare candidate RL algorithms suited for continuous, potentially high-dimensional actions [9†L37-L40] [16†L84-L93]. Below are typical choices, with pros/cons:

Algorithm	Type	Continuous Action?	Sample Efficiency	Stability/Tuning	Notes
Deep Q-Network (DQN) [14†L44-L53]	Off-policy, Value-based	No (discrete only)	Moderate	Sensitive (experience replay needed)	Baseline value-based; must discretize actions (impractical here).
DQN with discretization	Off-policy	Approximated	Low (large space)	Even harder (combinatorial)	Poor choice: action space continuous, K large.
DDPG [38†L30-L39] (Deterministic Policy Grad.)	Off-policy, Actor-Critic	Yes (Deterministic)	Medium	Unstable, requires careful tuning	Gives continuous control but known to diverge.

Algorithm	Type	Continuous Action?	Sample Efficiency	Stability/Tuning	Notes
TD3 (Twin Delayed DDPG)	Off-policy, Actor-Critic	Yes	Medium	More stable than DDPG	Improves DDPG with clipped noise, delayed updates.
Soft Actor-Critic (SAC) [24†L37-L44]	Off-policy, Actor-Critic, MaxEnt	Yes	High (sample-efficient)	High (robust, stable)	State-of-art for continuous; stable across seeds.
Proximal Policy Optimization (PPO) [20†L33-L40]	On-policy, Actor-Critic	Yes	Moderate	Good (simple clipping, easy)	Simpler to implement; but less sample-efficient.
A2C/A3C (Advantage Actor-Critic)	On-policy	Yes	Low	Decent (multi-threaded)	Can work, but PPO generally outperforms A2C.

- **DQN** [14†L44-L53] cannot directly handle continuous p, w without discretizing the action space (impractical for 64 dimensions).
- **DDPG/TD3** [38†L30-L39] allow continuous actions but often require extensive tuning (learning rates $\sim 1e-3$, large replay memory, target networks). TD3 (clipped double Q-learning) is an improvement to stabilize DDPG.
- **Soft Actor-Critic (SAC)** [24†L37-L44] maximizes entropy in the objective, yielding highly sample-efficient and robust training. It has shown state-of-art results in many continuous control tasks (often outperforming on-policy methods) [24†L37-L44]. SAC is stable across random seeds and has few hyperparameters (entropy temperature, learning rates).
- **Proximal Policy Optimization (PPO)** [20†L33-L40] is an on-policy policy-gradient method. It uses a clipped surrogate objective, is simpler to implement, and is stable. It often matches SAC performance on moderate-sized tasks, though it requires fresh samples every update (poorer sample efficiency). PPO may be easier to get working quickly.
- Other options: **Model-based RL** (learning a transition model) is advanced and not needed initially. **Multi-agent RL** is not required unless multiple decision-makers exist.

Hyperparameters: For on-policy methods (PPO/A2C), common defaults are: learning rate $\approx 10^{-4}$, batch/epoch sizes of 1024–4096 steps, clipping $\epsilon \approx 0.2$, discount $\gamma = 0.99$, GAE $\lambda = 0.95$. For off-policy (SAC/TD3): replay buffer of $\sim 1e6$ transitions, batch size ~ 256 , learning rate $\sim 3e-4$, target update rate $\tau = 0.005$. These serve as starting points; we will tune via grid or Bayesian search.

```
# Example (initial) hyperparameter table:
- PPO:
```

- optimizer: Adam
- lr: 3e-4
- batch: 2048
- clip: 0.2
- gamma: 0.99
- lam: 0.95
- SAC:
 - lr (actor/critic): 3e-4
 - batch: 256
 - replay_size: 1e6
 - gamma: 0.99
 - ent_coef: auto (or 0.2)
 - target_smoothing: 0.005

A comparative pros/cons summary:

- *PPO*: stable, easy, well-tested in many tasks [20†L33-L40] . Lower sample efficiency but robust learning curves.
- *SAC*: highly sample-efficient off-policy, handles exploration via entropy [24†L37-L44] . More complex code but widely available (Stable-Baselines3, RLlib).
- *DDPG/TD3*: older baseline; simpler actor-critic but prone to instability (especially with many action dims).

We will likely start with PPO and SAC as primary contenders. (For instance, Fu et al. successfully used SAC for a similar ISAC power-allocation MDP [36†L66-L74] .) If PPO converges and yields good performance, it may suffice. Otherwise, SAC’s off-policy training can leverage simulation steps more efficiently.

4. Integration with Existing Model and Data

Our RL agent must integrate with the current project’s simulation and data pipelines:

- **Model Architecture:** The current system uses a waveform model with K subcarriers (Table 1: $K=32$) and computes R and P_d via known formulas [?] [36†L66-L74] . We can reuse these computations in the environment reward function and state observations. For example, given p, w , use the same rate formula (likely Shannon-like per subcarrier sum) and the Gaussian Q-function for detection (Eq.4). This ensures consistency.
- **Datasets:** The project has generated a limited “surrogate analysis” dataset (80 scenarios $\times$ features) for XGBoost [?] . That offline dataset is small; instead, RL will generate new data by interacting with the simulator. If desired, we could use the existing data for offline pretraining or as a check, but RL training will not directly use it. (Offline RL is possible, but unnecessary if we can simulate.)
- **Offline vs Online:** We assume an *online* RL setting: the agent iteratively interacts with the environment. However, since we have full control, we can do batched learning (collect many episodes before updating) as needed. If the simulation is slow, we can parallelize episodes (see compute).
- **Data Pipeline:** At each step the environment emits a tuple (s, a, r, s') . We may store these in a *replay buffer* (for off-policy methods). We should implement efficient data handling (e.g. NumPy buffers, or

use an RL library's built-in buffer). We also need to handle normalization of inputs (e.g. scale channel gains) and clipping of rewards if extreme.

- **Existing Model Use:** The original model (GWO, XGBoost) is not part of the RL training. However, we should define baselines by comparing the RL policy's performance against the best static solution found by GWO on sample episodes. E.g., freeze a trained RL policy and evaluate its J score on the same scenarios the GWO solved.

Offline data possibility: If we wanted an offline RL approach, we could treat the existing 80-sample dataset as an "experience replay" for initial training. But this is very limited data; better to simulate more. Offline RL (such as Conservative Q-Learning) could be mentioned as an advanced option but is not needed initially.

Overall, the integration plan is to embed the RL environment in the project's codebase, wrapping the channel/power computations in the Gym API, and then link the agent code (e.g. using Stable-Baselines3 or a custom PyTorch script) to call `env.step(action)` in training loops.

5. Training Pipeline and Compute Requirements

We recommend using standard RL libraries to build the training pipeline. For example, [Stable-Baselines3 \(SB3\)](#) [28†L67-L74] or Ray RLlib can handle common algorithms (PPO, SAC, etc.), logging, and GPU acceleration.

Hardware: We anticipate using a GPU for neural network training (especially for larger policy/value networks) and CPUs for parallel environment simulation. SAC (off-policy) can run on a single GPU and use vectorized environments (e.g. 8 parallel envs) to generate experience quickly. PPO (on-policy) benefits from many CPU cores to collect batches. For moderate-sized neural nets (MLPs with a few layers), 8–16 GB GPU memory is sufficient. If budgets allow, two GPUs can speed up experimentation (one for PPO, one for SAC, or hyperparameter sweeps).

Memory: The replay buffer for SAC/TD3 may hold up to $\sim 1e6$ transitions (each transition storing two vectors of length $\sim 2K$ and a scalar reward). At $K=32$, each transition might be ~ 256 bytes, so 10^6 transitions ~ 256 MB. Networks (two hidden layers of 256 units) are small (<10 MB). Total memory <1 GB.

Compute Time: RL can be sample-inefficient. Based on similar tasks [24†L37-L44] [20†L33-L40], training might require on the order of **0.5–3 million environment steps** to converge. If each step (including channel simulation and network forward/backprop) takes ~ 1 ms on GPU+CPU, $3e6$ steps ~ 3000 seconds (~ 1 hour). However, with overhead and multiple training epochs, expect **hours to days** of training per experiment. PPO often takes longer wall-clock (due to on-policy rollouts) than SAC for the same number of gradient updates. We should budget *several days* on a mid-range GPU for initial training runs, and more for hyperparameter tuning.

Component	Resource Estimate
Env simulation	~ 0.1 – 1 ms per step (vectorizable)
Neural nets	2-3 layers MLP, $\sim 50k$ parameters
Training steps	~ 1 – 5 million steps per agent run

Component	Resource Estimate
GPU usage	1× NVIDIA RTX 3070 (8 GB) suffices
Parallelization	Use ~8 envs for PPO, ~1-4 for SAC
Disk/Storage	Store checkpoints (~100 MB each) and logs.

Training Loop Sketch

A typical training loop (for off-policy SAC/TD3) is:

```

envs = VectorizedEnvs(n=8)
replay_buffer = ReplayBuffer(max_size=1_000_000)

# Initialize agent (policy & value networks)
agent = SACAgent(state_dim, action_dim)

for epoch in range(num_epochs):
    for step in range(steps_per_epoch):
        actions = agent.sample_action(states)          # e.g., from Gaussian
        policy
        next_states, rewards, dones, _ = envs.step(actions)
        replay_buffer.store(states, actions, rewards, next_states, dones)
        states = next_states

        if replay_buffer.size() > start_training:
            batch = replay_buffer.sample(batch_size=256)
            agent.update(batch) # perform gradient update (actor & critic)

# Optionally evaluate policy, log metrics, save checkpoints
eval_metrics = evaluate_policy(agent.policy, envs_eval)
log_metrics(epoch, eval_metrics)

```

For on-policy PPO, the loop alternates collecting trajectories and then updating:

```

envs = VectorizedEnvs(n=16)
agent = PPOAgent(state_dim, action_dim)

for update in range(num_updates):
    batch_states, batch_actions, batch_rewards, batch_dones =
collect_trajectories(envs, agent, horizon)
    agent.update_policy(batch_states, batch_actions, batch_rewards, batch_dones)
    log_metrics(update, ...)

```

In either case, we will enable **TensorBoard logging** (via SB3 or PyTorch) to monitor episode returns, actor/critic losses, and hyperparameters. Checkpoints of model weights should be saved periodically to allow restarts.

6. Evaluation Protocol and Metrics

Evaluation of the RL agent must be rigorous and comparable to existing baselines. We propose:

- **Metrics:** Use the same metrics as the paper: average communication *rate* (*bps/Hz*), average detection probability P_d , and the weighted objective J with chosen α [36†L66-L74] [?†]. Compute these on held-out test scenarios (new random channel sequences). We may also track variance across episodes.
- **Baselines:** Compare against the state-of-the-art from the original study: GWO (Gray Wolf Optimizer) allocation, PSO, DE, etc., as well as trivial baselines (e.g., equal power, flat waveform). For example, run GWO on the same test episodes to get “optimal” J for reference (using the second paper’s code or saved solutions if available). Also compare to an SNR-based allocation (the original method) if applicable. The prior static solutions (Table 4 in paper) give a reference.
- **Learning Curves:** Plot cumulative reward and metrics vs. training iterations to ensure learning is occurring. Check if performance plateaus.
- **Ablation Studies:** Evaluate the impact of design choices:
 - Vary α to see if policy adapts (e.g., more weight on detection vs comm).
 - Try different network architectures or reward scalings.
 - Compare on-policy (PPO) vs off-policy (SAC) results.
- **Statistical Validation:** Perform multiple runs (3–5 random seeds) of each algorithm to account for variance. Report mean $\pm$ standard deviation of key metrics. If comparing methods, use statistical tests (e.g. t-test) to confirm significance of any improvements.
- **Timeline:** Measure wall-clock time to train each algorithm to convergence for fair compute comparisons.

All performance comparisons should cite the RL literature or project references when describing methods (e.g. citing [20] or [24] when discussing sample complexity observations). Any new metric (e.g. average reward) should be clearly defined.

7. Debugging, Monitoring, and Reproducibility

Reliable experimentation requires careful practices:

- **Logging:** Use TensorBoard or Weights & Biases to log training curves: episode return, loss values, learning rates, hyperparameters. Plot also environment-specific stats (e.g. average power usage). Save visualizations periodically.
- **Checkpoints:** Save model checkpoints (policy/critic weights) frequently. On crash, resume from last checkpoint. Also save replay buffer states for reproducibility if needed.
- **Random Seeds:** Fix random seeds for NumPy, PyTorch, Gym, etc., in each run and document them. This ensures experiments are reproducible [32†L149-L157] [28†L67-L74]. Record the seed along with results.
- **Unit Tests:** Write unit tests for the environment (e.g. reward calculation matches expected values for known inputs) and for the agent’s update steps. Verify that gradient updates are not NaN/Inf.

- **Debugging Tools:** Use smaller environments for debugging: e.g., reduce K , power, or horizon to simplify. Check that reward increases when agent follows sensible heuristics. If training fails (no improvement), visualize sample trajectories.
- **Hyperparameter Sweeps:** Automate hyperparameter tuning (e.g. using wandb or simple grid search) and track results.

These practices follow standard RL guidelines [28†L67-L74] [32†L149-L157] . They ensure that results can be replicated and that model performance can be traced back to seeds and settings.

8. Risks, Failure Modes, and Mitigation

Convergence Failure: RL may fail to learn (stuck in local optima or oscillating). Mitigation: start with a stable algorithm (PPO or SAC). Use reward shaping (e.g. normalize R and P_d) to ensure balanced gradients. If unstable, reduce network size or learning rate [16†L84-L93] [20†L33-L40] .

Exploration Issues: The agent might allocate all power to one subcarrier or extreme waveform. Use entropy regularization (as in SAC) or add exploration noise (in DDPG) to encourage diverse actions initially. Also clipping rewards or using a slight discount can stabilize learning.

Action Constraint Violations: Handling the sum constraints incorrectly could produce invalid actions (e.g. sum of powers $\neq$ budget). Implement projection layers or use action parameterizations (e.g. Dirichlet policies [36†L66-L74]) to enforce feasibility.

Overfitting to Channel Models: The agent might overfit to the assumed channel statistics. Mitigate by training on diverse channel realizations (different scenarios, seeds). Validate on out-of-sample channel models.

Compute Limitations: Training can be compute-intensive. Use parallelism wisely (vectorized envs for sample efficiency). Monitor GPU utilization; if training is slow, consider simpler networks. Early stopping or curriculum learning (start with easier tasks) can reduce time.

Comparison Fairness: Ensure RL baselines are fairly tuned. Compare only final converged performance, and use equal compute budgets where possible.

By anticipating these issues and adopting practices like reward normalization and stable algorithms (PPO/SAC) [20†L33-L40] [24†L37-L44] , we mitigate risks of failure.

9. Implementation Roadmap and Milestones

We propose a phased plan with clear milestones and approximate durations:

Milestone	Duration	Description
1. Extract Project Specifications	1 wk	Collect details from second paper: state variables, actions, reward, horizon, channel models.

Milestone	Duration	Description
2. Define RL Formulation	1 wk	Finalize MDP: state/action space, reward function, episodic structure. Document assumptions and required inputs.
3. Build Simulation Environment	2 wks	Implement Gym-style <code>ISACEnv</code> with channel model, reward, and dynamics. Validate against original code (e.g. reproduce static cases).
4. Baseline Implementation	1 wk	Integrate GWO or PSO from original project as comparison policies. Prepare evaluation scripts.
5. Implement RL Algorithms	3 wks	Code or integrate (SB3) PPO and SAC agents. Set up training loops, logging. Ensure code structure (see Section 10).
6. Preliminary Training Runs	2 wks	Run PPO and SAC on simple scenarios (single scenario, short horizon) to verify learning. Tune key hyperparameters.
7. Full-Scale Training	3 wks	Train agents on full dynamic scenarios (multiple channel types). Collect multiple runs.
8. Evaluation and Analysis	2 wks	Compare RL policies to baselines on test scenarios. Perform ablations (vary α , horizon). Run statistical tests.
9. Refinement and Ablations	2 wks	Adjust design as needed (e.g. try TD3, adjust network). Include any additional features (curriculum, reward tweaks).
10. Documentation and Prototype Release	2 wks	Finalize code, write up results. Prepare minimal viable demo (e.g. Jupyter notebook).

Total estimate: ~17 weeks (approx. 4 months). The MVP by end of step 5 should be a working agent on a toy scenario. We would then iteratively improve performance and analysis in steps 6–9.

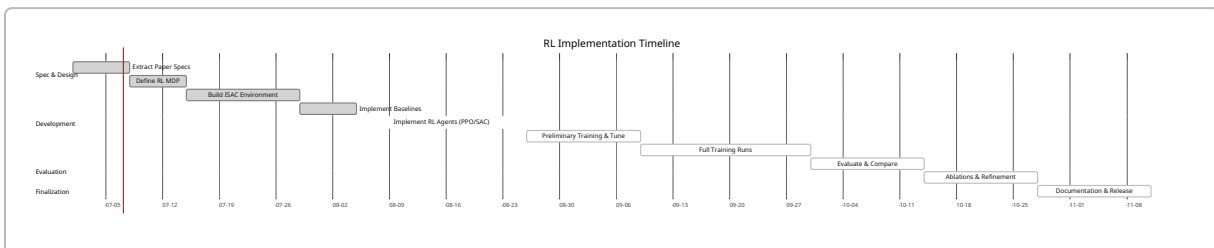


Figure: *Proposed implementation roadmap (Gantt chart)*. Shaded bars indicate completed/planned tasks with approximate durations. Each milestone builds on the previous one.

10. Code Structure and Pseudocode

A clean modular codebase is crucial. We suggest the following structure (in a Python project):

```

project_root/
├─ envs/
│   └─ isac_env.py          # Gym-compatible environment class
├─ agents/
│   ├── ppo_agent.py        # PPO agent implementation (or SB3 wrapper)
│   └─ sac_agent.py         # SAC agent implementation
├─ train.py                 # Training script orchestrating env and agent
├─ evaluate.py              # Evaluation script (runs trained models on test
cases)
├─ utils/
│   ├── replay_buffer.py    # Experience replay for off-policy methods
│   ├── config.yaml        # Hyperparameters and settings
│   └─ monitoring.py        # Logging (TensorBoard/WandB) setup
├─ data/
│   └─ ...                  # (Optional) saved experience or results
├─ tests/
│   └─ test_env.py          # Unit tests for environment dynamics/reward
└─ docs/
    └─ report.md            # This analysis report

```

Key function signatures:

- `ISACEnv.reset() -> state`: initializes and returns first observation.
- `ISACEnv.step(action: np.array) -> (state, reward, done, info)`: applies action, simulates channel update, computes reward.
- `PPOAgent.learn(env, config)`: trains the policy using PPO on given env.
- `SACAgent.learn(env, config)`: similarly for SAC.
- `evaluate(agent, env, num_episodes) -> metrics_dict`: runs episodes and returns averaged metrics.

Example Pseudocode (Training Loop, PPO)

```

env = ISACEnv(config)
agent = PPOAgent(state_dim, action_dim, config) # could wrap SB3.PPO

for iteration in range(num_iterations):
    trajectories = [] # collect batch of episodes
    for ep in range(batch_size):
        state = env.reset()
        done = False
        ep_data = []
        while not done:
            action = agent.select_action(state) # stochastic policy
            next_state, reward, done, _ = env.step(action)
            ep_data.append((state, action, reward, next_state, done))

```

```

        state = next_state
        trajectories.extend(ep_data)

    agent.update(trajectories) # perform PPO update on collected batch

# Logging, evaluation
if iteration % log_interval == 0:
    metrics = evaluate(agent.policy, test_env)
    log(metrics, iteration)

```

Key Design Notes:

- Use a **common interface** for agents (e.g. an abstract `Agent` class) so that different algorithms (PPO, SAC, etc.) can be swapped easily.
- Use **vectorized environments** (e.g. `SubprocVecEnv` from Stable-Baselines3) to parallelize data collection. This significantly speeds up on-policy methods.
- Ensure **seed control** by passing a `seed` parameter to envs and PyTorch.

All network models and replay buffers can rely on PyTorch or TensorFlow. We recommend PyTorch for flexibility and wide community support.

11. Recommended Libraries, Frameworks, and References

Libraries/Frameworks:

- **Stable-Baselines3 (SB3)** [28†L67-L74] : well-tested PyTorch implementations of PPO, SAC, DDPG, TD3, etc. Handles many implementation details (policy/value networks, buffers) out-of-the-box.
- **Gymnasium** [32†L149-L157] : standard RL environment API for Python, enabling easy integration with SB3 and other libraries.
- **RLlib (Ray)**: for scalable, production RL, though SB3 is sufficient for research.
- **TensorBoard / WandB**: for experiment logging and visualization.
- **PyTorch**: for custom neural networks if needed.
- **NumPy / SciPy**: for environment calculations (SNR, Gaussian functions, etc.).

Primary Sources and References:

- **Alwarafy et al. (2022)** [9†L37-L45] : Survey on DRL for wireless resource allocation, motivating RL in our setting.
- **Mnih et al. (2015)** [16†L84-L93] : Introduction of DQN and fundamental RL concepts (Q-learning with function approximation).
- **Lillicrap et al. (2015)** [38†L30-L39] : DDPG for continuous control.
- **Schulman et al. (2017)** [20†L33-L40] : PPO algorithm description.
- **Haarnoja et al. (2018)** [24†L37-L44] : Soft Actor-Critic algorithm.
- **Fu et al. (2026)** [36†L66-L74] : Recent work using SAC for ISAC power allocation (motivating reward design).
- **OpenAI Gym documentation** [32†L149-L157] : Gymnasium interface for environment implementation.

- **Stable-Baselines3 documentation** [28†L67-L74] : reliable implementations of RL algorithms.

Each of the above sources provides guidance: e.g. [24] shows SAC's stability claims, [20] highlights PPO's simplicity [20†L33-L40] , and [36] demonstrates modeling of ISAC allocation as an MDP with SAC.

With this guide, a practitioner can systematically implement and evaluate RL for the ISAC power-waveform problem. The roadmap and code sketches provide a blueprint for development, while the references ensure adherence to best practices in RL and wireless communications.
